SOFTWARE ENGINEERING
& PROJECT MANAGEMENT

DAY 3

The Waterfall Model

A student-facing chapter with practical examples, diagrams, strengths, limitations, and classroom activities

Module I: Introduction to Software Engineering
Topic: Software Process Models — Waterfall Model
Duration: 1 hour

Running example: College Event Management System (CEMS)

Use this chapter to understand how a software project can move through a sequence of planned phases, what each phase produces, and why the model is useful in some projects but risky in others.

1. Learning Objectives

By the end of this chapter, you should be able to:

- Define a software process model and explain the purpose of the Waterfall Model.
- Describe the main phases of Waterfall in the correct order.
- Identify the key activities and deliverables in each phase.
- Apply Waterfall to a realistic software project such as a College Event Management System.
- Explain the advantages, disadvantages, and suitable use cases of the model.
- Compare a change request against the planned sequence of work.

> **Key idea**
> The Waterfall Model organizes development into a sequence of phases. A phase is generally completed and reviewed before the project proceeds to the next phase.

2. What Is a Software Process Model?

A **software process model** is a structured way of organizing the activities used to build and maintain software. It describes the major stages of work, how those stages relate to one another, and what outputs are expected from them.

Imagine constructing a college building. The team first studies requirements, prepares drawings, estimates cost, constructs the building, inspects it, and finally hands it over. Software is not a physical building, but a planned process also helps a software team coordinate work, check quality, manage risk, and communicate progress.

Common process models in the syllabus include Waterfall, Incremental, Spiral, Prototype, and Agile. Each organizes work differently and suits different circumstances.

3. The Waterfall Model

The **Waterfall Model** is a sequential software development model in which work proceeds through a series of defined phases. The output of one phase becomes an input to the next phase. Teams normally finish and document one phase before moving forward.

It is called “Waterfall” because a simplified diagram shows work flowing downward from one phase to the next, like water flowing down steps.

1. Requirements Analysis
↓
2. System Design
↓
3. Implementation / Coding
↓
4. Integration & Testing
↓
5. Deployment

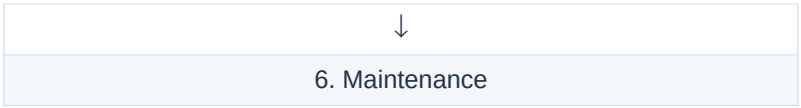

Figure 1. A simplified Waterfall sequence. Real projects may include reviews, feedback, and repeated work when defects or change requests are discovered.

4. Phases of the Waterfall Model

The names and exact boundaries of phases can vary by organization. The following sequence is a practical version for learning and project planning.

Phase 1 — Requirements Analysis

Purpose: Understand what the customer or users need before detailed design and coding begin.

- Meet stakeholders such as students, faculty coordinators, event organizers, and college administrators.
- Record functional requirements: users can create events, register, view schedules, and check attendance.
- Record non-functional requirements: the system should be secure, usable, reliable, and able to handle expected traffic.
- Resolve ambiguity and agree on scope, assumptions, and acceptance criteria.

Deliverables: Software Requirements Specification (SRS), approved scope, and requirement list.

Example: “Students must be able to register for an event until registration closes.” This is clearer than “registration should be easy.”

Phase 2 — System Design

Purpose: Decide how the system will meet the requirements. The team plans the architecture, modules, database, interfaces, and security approach.

- Choose modules such as Login, Event Management, Registration, Notifications, and Attendance.
- Design database entities such as User, Event, Registration, and Attendance.
- Define interfaces, permissions, data flow, and integration points.
- Prepare diagrams and design specifications for developers.

Deliverables: Architecture/design document, database schema, interface designs, and relevant UML diagrams.

Phase 3 — Implementation / Coding

Purpose: Developers convert the approved design into working code.

- Create the agreed modules and follow coding standards.
- Use version control, such as Git, to track changes.
- Write unit tests and review code where required.
- Keep implementation aligned with the approved design and requirements.

Deliverables: Source code, build artefacts, unit tests, and code review records.

Phase 4 — Integration and Testing

Purpose: Check whether the software works correctly and satisfies the specified requirements.

- Integrate modules such as event creation, registration, and notifications.
- Test valid and invalid inputs, user permissions, and edge cases.
- Record defects, fix them, and retest affected functionality.
- Confirm that the product meets the agreed acceptance criteria.

Deliverables: Test plan, test cases, defect reports, test results, and release recommendation.

Phase 5 — Deployment

Purpose: Release the tested software to the intended users or production environment.

- Prepare the production environment and configuration.
- Migrate or initialize data where necessary.
- Provide user instructions or training.
- Plan release communication and a rollback approach for serious issues.

Deliverables: Deployed application, release notes, user guide, and deployment record.

Phase 6 — Maintenance

Purpose: Support the software after release by correcting problems and handling approved changes.

- Correct defects found in production.
- Adapt the software when browsers, operating systems, or external services change.
- Improve performance, usability, or reliability.
- Assess change requests for impact, cost, schedule, and risk.

Deliverables: Patches, updated documentation, change records, and new versions.

5. Worked Example: College Event Management System

A college wants a web application to publish events, accept student registrations, and let coordinators record attendance. The project sponsor has approved a clear initial scope and a fixed deadline.

Phase	What the team does	Example output
Requirements	Interview students and event coordinators; agree on features and rules.	Approved SRS
Design	Plan pages, modules, database tables, roles, and access controls.	Design document and schema
Implementation	Build login, event listing, registration, and attendance modules.	Working code
Testing	Check registration limits, duplicate sign-ups, permissions, and attendance records.	Test results and defect log
Deployment	Publish the application and provide instructions to coordinators.	Live system and release notes
Maintenance	Fix production defects and evaluate later feature requests.	Patch or planned update

Why the sequence matters
If the team starts coding before agreeing on the registration rules, developers may implement different assumptions. A requirements review aims to discover and resolve those differences before they become expensive rework.

6. Change Request: QR-Code Attendance

Suppose that after design and coding have begun, the college asks for QR-code attendance instead of manual attendance.

In a Waterfall project, this is not simply a casual addition. The team should log the change, clarify the requirement, analyze its impact, estimate additional effort, identify design and test changes, and obtain approval through the project's change-control process.

1. Record the change request and the reason for it.
2. Analyze impact on requirements, design, database, security, cost, and schedule.
3. Decide whether to approve, reject, or defer the request.
4. If approved, update the relevant documents and plans.
5. Implement, test, and release the change using the agreed process.

Learning point
Waterfall does not mean that change is impossible. It means that late changes can be more disruptive because earlier phases and documents may need to be revisited.

7. Advantages of the Waterfall Model

- **Clear structure:** Phases and milestones are easy to understand.
- **Defined deliverables:** Each phase produces documents or outputs that can be reviewed.
- **Progress tracking:** Managers can compare completed milestones with the plan.
- **Useful documentation:** Records help with handover, maintenance, and audits.
- **Suitable for stable scope:** It can work well when requirements are understood and unlikely to change significantly.

8. Limitations of the Waterfall Model

- **Late feedback:** Users may not see a working product until relatively late.
- **Costly late changes:** A changed requirement can affect documents, design, code, and tests.
- **Risk of misunderstanding:** If the team misunderstands requirements early, the mistake may travel through later phases.
- **Limited flexibility:** A rigid sequence can be uncomfortable in projects where needs evolve frequently.
- **Delayed discovery:** Some usability or technical problems may become obvious only after implementation.

9. When Is Waterfall a Reasonable Choice?

Waterfall may be a reasonable choice when requirements are stable, the scope is well understood, approvals and documentation are important, and the technology is familiar. Examples can include a clearly specified internal reporting tool or a system upgrade with a fixed and well-understood scope.

It may be a weaker choice for a new consumer app whose users are still discovering what they need, or for a product facing frequent market changes. Such projects may benefit from iterative or incremental approaches.

> **Avoid an absolute rule**
> No process model is best for every project. The right choice depends on requirement stability, risk, customer involvement, regulation, team capability, and delivery needs.

10. Waterfall Compared with an Iterative Approach

Question	Waterfall	Iterative / Incremental approach
How is work organized?	Main phases are completed in sequence.	Work is repeated in cycles or delivered in portions.
When can users give feedback?	Often after major work has progressed or a release is available.	Can be earlier and more frequent.
Handling new information	May require formal re-planning and revisiting earlier outputs.	Can be incorporated into a later iteration or increment.
Potential fit	Stable, clearly specified requirements.	Evolving requirements or value that can be delivered in portions.

11. Classroom Activity: Choose the Process

Work in pairs. For each scenario, decide whether Waterfall seems suitable and explain your reasoning in two or three sentences.

1. A college must replace an old fee-reporting tool. The report fields, calculations, users, and approval rules have already been documented and approved.
2. A start-up is building a new social app. User feedback is expected to change the feature list every few weeks.
3. A government contract requires formal approvals, traceable requirements, extensive documentation, and a clearly specified deliverable.

Discussion prompts: Are the requirements stable? How often is feedback needed? What is the impact of a late change? Which project constraints matter most?

12. Knowledge Check

Answer these questions without looking at the chapter, then check your understanding.

1. What is a software process model?
2. List the main Waterfall phases in order.
3. What is one deliverable from the requirements phase?
4. Why can a late change be expensive in Waterfall?
5. State two advantages and two limitations of Waterfall.
6. Give one example of a project where Waterfall may be suitable and explain why.
7. Does Waterfall make change impossible? Explain.
8. Which phase focuses on checking whether the system meets its requirements?

Answer key

1. A structured approach for organizing software development activities. 2. Requirements, design, implementation, integration/testing, deployment, maintenance. 3. An approved SRS or requirement list. 4. It can affect earlier documents, design, code, and test work. 5. Examples: clear milestones/documentation; late feedback/costly changes. 6. A stable, well-specified project. 7. No; changes can be managed, but may require formal impact analysis and rework. 8. Testing.

13. Key Terms

Term	Meaning
Process model	A structured way to organize software development activities.
Waterfall Model	A sequential model that organizes development into defined phases.
SRS	Software Requirements Specification; a document describing agreed requirements.
Deliverable	An output produced by a project activity or phase.
Verification	Checking work products against specified requirements or standards.
Validation	Checking whether the resulting system meets user needs.
Change request	A formal proposal to modify an agreed requirement, design, or product.
Maintenance	Work performed after release to correct, adapt, or improve software.

14. Homework / Preparation for Day 4

Prepare a one-page Waterfall analysis for a Library Management System. Include:

- Three functional requirements and two non-functional requirements.
- The six main Waterfall phases and one key deliverable from each.
- Two advantages and two limitations for this project.
- One likely change request and a short impact analysis.

Next chapter
Day 4 will explore the Incremental Model: how a system can be delivered in useful portions instead of waiting for the entire product to be completed.

Syllabus alignment: Module I — Introduction to Software Engineering; Software Process Models: Waterfall. This chapter uses the College Event Management System as an illustrative teaching example.